Ch 1 – Mathematical Preliminaries (Lec 1)

Sets

Def. set collection of elements. $1 \in A$, ship $\notin B$. Representations: explicit, ellipsis, set-builder $S = \{j : j > 0,\ j = 2k,\ k > 0\}$. Operations: $A \cup B$, $A \cap B$, $A - B$ (e.g. $\{1,2,3\} - \{2,3,4,5\} = \{1\}$), complement $\bar{A} = U - A$. $\bar{\bar{A}} = A$. **DeMorgan:** $\overline{A \cup B} = \bar{A} \cap \bar{B}$; $\overline{A \cap B} = \bar{A} \cup \bar{B}$. $\emptyset = \{\}$; $S \cup \emptyset = S$, $S \cap \emptyset = \emptyset$, $S - \emptyset = S$. **KEY:** $\emptyset \neq \{\lambda\}$: $|\{\}| = 0$, $|\{\lambda\}| = 1$, $|\lambda| = 0$. Subset $A \subseteq B$ (proper $\subset$). Disjoint: $A \cap B = \emptyset$. Cardinality $|A|$. Powerset $2^S = \mathcal{P}(S) = \{\text{all subsets}\}$; if S finite then $|2^S| = 2^{|S|}$. **Ex. 1.1** $S = \{a,b,c\}$: $2^S = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\}\}$, $|2^S| = 2^3 = 8$. Cartesian product $A \times B = \{(x,y) : x \in A,\ y \in B\}$, $|A \times B| = |A||B|$. Order matters: $(4,2) \in A \times B$ but $(2,4) \notin A \times B$. Partition of S: $S_1, \ldots, S_n$ mutually disjoint, union $= S$, all nonempty.

Functions

$f : A \to B$ assigns each domain element a *unique* range element. Total if defined on whole A; else partial.

Asymptotic Analysis

$f = O(g)$: $\exists c, n_0 > 0$: $0 \leq f(n) \leq c\,g(n)\ \forall n \geq n_0$ (upper). $f = \Omega(g)$: $0 \leq c\,g(n) \leq f(n)$ (lower). $f = \Theta(g)$ iff both O and Ω (tight). **Ex. 1.3** $f = 2n^2 + 3n$, $g = n^3$, $h = 10n^2 + 100$: $f = O(g)$, $g = \Omega(h)$, $f = \Theta(h)$.

Relations

Relation: a set of pairs, $x_i\,R\,y_i$; one x may relate to many y (unlike functions). Equivalence relation: **reflexive** (xRx), **symmetric** ($xRy \Rightarrow yRx$), **transitive** ($xRy \wedge yRz \Rightarrow xRz$). **Ex. 1.4** $x \equiv y$ iff $x \bmod 3 = y \bmod 3$ on $\mathbb{N}$: $2 \equiv 5$, $12 \equiv 0$, $0 \equiv 36$; an equivalence (equivalence classes: residues mod 3).

Graphs

Digraph $G = (V, E)$: nodes V, edges $E \subseteq V \times V$ (ordered pairs). **Def. walk** sequence of adjacent edges. **Def. path** walk with no edge repeated; *simple path*: no node repeated. **Def. cycle** walk from a node back to itself; *simple cycle*: only base node repeated. **Def. loop** edge from vertex to itself. **Euler tour:** cycle containing each *edge* once. **Hamiltonian cycle:** simple cycle containing all *nodes*. Finding all simple paths: grow from origin, depth-first, never repeat a node. **Def. tree** graph with no cycles. root, parent/child, leaf, level (root = level 0), height = longest root–leaf path.

Proof Techniques

1. **Direct/constructive:** assume X, show Y.
2. **Induction:** basis $P_1, \ldots, P_b$ true; hypothesis $P_1, \ldots, P_k$; step: show P_{k+1}. **Ex. 1.5** Binary tree of height n has $\leq 2^n$ leaves. Let $L(i) =$ max leaves at height i. Basis $L(0) = 1$. Hyp. $L(i) \leq 2^i$, $i \leq k$. Step: each level-k leaf adds at most 2 children: $L(k+1) \leq 2L(k) \leq 2 \cdot 2^k = 2^{k+1}$. (Note: recursion, e.g. $f(n) = f(n-1) + f(n-2)$, is a different thing.) **Ex. 1.6** n mutually intersecting lines divide plane into $A(n)$ regions; $A(1) = 2, A(2) = 4, A(3) = 7$, $A(n+1) = A(n) + n + 1$; solved form $A(n) = \frac{n(n+1)}{2} + 1$.
3. **Contradiction:** assume P false, derive impossible conclusion. **Ex. 1.7** $\sqrt{2} \notin \mathbb{Q}$: assume $\sqrt{2} = n/m$ coprime; $2m^2 = n^2$ so n even, $n = 2k$; $2m^2 = 4k^2 \Rightarrow m^2 = 2k^2$ so m even; n, m share factor 2 – contradiction.

Languages & Grammars (three basic concepts: languages, grammars, automata)

Alphabet Σ (finite symbols); string $w = a_1 \cdots a_n$ over Σ; language = any subset of Σ^*.

- Concatenation wv; Reverse $w^R = a_n \cdots a_1$; Length $|w|$, $|uv| = |u| + |v|$.
- Empty string λ: $|\lambda| = 0$, $\lambda w = w\lambda = w$.
- Substring = consecutive chars; prefix/suffix: $w = uv$ (u prefix, v suffix).
- Power $w^n = ww \cdots w$ (n times), $w^0 = \lambda$; $(abba)^2 = abbaabba$.
- Σ^* = all strings over Σ (incl. λ); $\Sigma^+ = \Sigma^* - \{\lambda\}$.

Language operations: union, intersection, difference, complement $\bar{L} = \Sigma^* - L$, reverse $L^R = \{w^R : w \in L\}$, concatenation $L_1L_2 = \{xy : x \in L_1, y \in L_2\}$, power L^n ($L^0 = \{\lambda\}$), **Kleene star** $L^* = L^0 \cup L^1 \cup L^2 \cup \cdots$, positive closure $L^+ = L^* - \{\lambda\}$. **Ex.** $\{a, ab, ba\}\{b, aa\} = \{ab, aaa, abb, abaa, bab, baaa\}$. **Ex.** $L = \{a^n b^n : n \geq 0\}$, then $L^2 = \{a^n b^n a^m b^m : n, m \geq 0\}$ – n, m *unrelated* ($aabbaaabbb \in L^2$).

Def. 1.1 grammar 4-tuple $G = (V, T, S, P)$: V variables, T terminals, $S \in V$ start variable, P productions $x \to y$ with $x \in (V \cup T)^+$, $y \in (V \cup T)^*$. **Derivation:** $\alpha \Rightarrow \gamma$ in one step iff $\alpha = \alpha_1 A \alpha_2$, $A \to \beta \in P$, $\gamma = \alpha_1 \beta \alpha_2$. $\alpha \overset{*}{\Rightarrow} \gamma$: zero or more steps. **Def. 1.2** $L(G) = \{w \in T^* : S \overset{*}{\Rightarrow} w\}$; $S, w_1, \ldots, w_n$ are *sentential forms*; w is a *sentence*. **Ex. 1.11** $S \to aSb \mid \lambda$: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$; $L(G) = \{a^n b^n : n \geq 0\}$. $aabb$ sentence, $aaSbb$ sentential form. **Ex. 1.12** Grammar for $L = \{a^n b^{n+1} : n \geq 0\}$: $S \to Ab$, $A \to aAb$, $A \to \lambda$ (one extra b). **Ex. 1.13** $S \to SS|\lambda|aSb|bSa$ generates $\{w : n_a(w) = n_b(w)\}$; proof by induction on $|w| = 2n$: $w = aw_1b$ (or bw_1a): $S \Rightarrow aSb \overset{*}{\Rightarrow} aw_1b$; else $w = w_1w_2$, $S \Rightarrow SS \overset{*}{\Rightarrow} w_1w_2$. **Equivalent grammars:** $L(G_1) = L(G_2)$. **Ex. 1.14** G_1: $S \to aSb|\lambda$; G_2: $S \to aAb|\lambda$, $A \to aAb|\lambda$ are equivalent (both $\{a^n b^n\}$).

Ch 2 – Finite Automata (Lec 2)

DFA

Def. 2.1 DFA $M = (Q, \Sigma, \delta, q_0, F)$: finite Q; input alphabet Σ; transition function $\delta : Q \times \Sigma \to Q$ (**total**, exactly one move per symbol); initial state $q_0 \in Q$; final states $F \subseteq Q$. **Def. 2.2 (extended δ^*)** $\delta^* : Q \times \Sigma^* \to Q$ recursively: $\delta^*(q, \lambda) = q$, $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$. **Thm. 2.1** There is a walk from q_i to q_j labeled w iff $\delta^*(q_i, w) = q_j$. **Def.** $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$ (regular language). $\bar{L}(M) = \{w : \delta^*(q_0, w) \notin F\}$. **Trap state:** non-final state with self-loops on all symbols (absorbs bad prefixes). **Ex. 2.2** $L = \{a^n b : n \geq 0\}$: $q_0 \overset{a}{\circlearrowleft}$, $q_0 \overset{b}{\to} q_1$ (final), $q_1 \overset{a,b}{\to} q_2$ trap. **Ex. 2.3** prefix ab: $q_0 \overset{a}{\to} q_1 \overset{b}{\to} q_2$ (final loop); trap q_3. **Ex. 2.4** without substring 001: states track longest 001-prefix seen: $\lambda, 0, 00, 001$ (trap). **Ex. 2.5** $L = \{awa : w \in \{a,b\}^*\}$ is regular (finite memory: remember first letter a; reject if other letters differ). **Ex. 2.6** $L^2 = \{aw_1aaw_2a\}$ likewise regular. **Non-regular** (preview): $\{a^n b^n : n \geq 0\}$ – needs unbounded counting; proved in Ch 4. **Regular language:** L is regular iff some DFA M has $L = L(M)$. **Design drills** (states = memory of what matters): exactly four 0s (counter 0–4+trap); exactly four 0s and ≥ 2 1s (5×3 grid); length ≤ 5; ends in 1101; binary value divisible by 5 (state = remainder mod 5); contains 0101; avoids 110. **Ex. Ex 2.1.7(a)** $|w| \bmod 3 = 0$: 3-state cycle on a, b, final state 0. **Ex. Ex 2.1.9(a)** every 00 followed immediately by 1: states $\lambda, 0, 00$; from 00 only 0 allowed; 00 then 1 returns. **Other drills:** starts with 0 and odd length, or starts with 1 and even length; every odd position is 1.

NFA

Def. $M = (Q, \Sigma, \delta, q_0, F)$ with $\delta : Q \times (\Sigma \cup \{\lambda\}) \to 2^Q$. Differences from DFA: range is powerset 2^Q; λ-transitions allowed (read head does not move); $\delta(q_i, a)$ may be $\emptyset$ (automaton *hangs*). **Accept:** string w accepted iff *some* computation consumes all of w and ends in a final state. **Reject:** *every* computation either ends non-final after consuming input, or hangs before input consumed. λ never appears on the input tape. **Def. 2.5** $\delta^*(q_i, w) =$ set of states reachable via a walk labeled w (λ-edges cost nothing). **Def. 2.6** $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}$. **Ex. 2.8** M: $q_0 \overset{1}{\to} q_1$, $q_1 \overset{0}{\to} \{q_0, q_2\}$, $q_0 \overset{\lambda}{\to} \{q_0, q_2\}$: $L(M) = \{10\}^*$; $\delta(q_2, 1) = \emptyset$. **Ex. 2.9** walks: $\delta^*(q_1, a) = \{q_0, q_1, q_2\}$; walk length may exceed $|w|$ (through λ-edges). **Ex.** $\{a^3\} \cup \{a^{2n} : n \geq 1\}$ is easy for an NFA (two branches), tedious for a DFA – nondeterminism gives concise descriptions, e.g. $S \to aSb|\lambda$. **Exercise 2.3.7:** any NFA $\to$ equivalent NFA with a *single* final state (add new final state, λ-edges from all old finals). Extreme case: NFA with no final state accepts $\emptyset$; add an unreachable final state.

Equivalence of DFA and NFA

Def. 2.7 M_1, M_2 equivalent iff $L(M_1) = L(M_2)$. **Thm. 2.2** Languages accepted by NFAs = regular languages (DFAs). Two directions: (1) DFA $\subseteq$ NFA: trivial (DFA is a special NFA). (2) NFA $\to$ DFA: **subset construction**:

1. Initial DFA state: $\{q_0\}$.
2. For DFA state $\{q_i, q_j, \ldots, q_m\}$ and symbol a: compute union of all $\delta^*(q_\ell, a)$ in NFA; add DFA transition $\delta'(\{q_i, q_j, \ldots\}, a) =$

$\{q_i', q_j', \dots\}$.
3. Repeat step 2 for all letters until no new states appear.
4. DFA state is final iff it *contains* an NFA final state.

DFA states $\in 2^Q$: $\emptyset, \{q_0\}, \{q_1\}, \{q_1, q_2\}, \dots$ (possibly exponentially many; at most $2^{|Q|}$). **Correctness proof (Thm 2.2):** induction on $|v|$: if in NFA $q_0 \overset{v}{\Rightarrow} q_m$, then in DFA $\{q_0\} \overset{v}{\Rightarrow} \{q_m, \dots\} \ni q_m$. Then $w \in L(M) \Rightarrow w \in L(M')$; converse similar; hence $L(M) = L(M')$.

Ex. 2.12 NFA $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{\lambda} q_2$, $q_2 \overset{b}{\circlearrowleft}$, q_1 final. DFA: $\{q_0\} \xrightarrow{a} \{q_1, q_2\}$ (final, since $q_1 \in F$); $\{q_0\} \xrightarrow{b} \emptyset$; $\{q_1, q_2\} \xrightarrow{a} \emptyset$, $\xrightarrow{b} \{q_2\}$; $\{q_2\} \xrightarrow{b} \{q_2\}$; $\emptyset$ loops on a, b. $L(M') = \{ab^*\} = L(M)$.

State Reduction (minimization)

Def. 2.8 States p, q of a DFA are **indistinguishable** if for *all* $w \in \Sigma^*$: $\delta^*(p, w) \in F \iff \delta^*(q, w) \in F$. Otherwise **distinguishable by** w (some w sends one to F, the other out). Algorithm (partition refinement):

1. Remove *inaccessible* states (unreachable from q_0).
2. Start: partition into F and $Q - F$ (final vs non-final). Every pair across this cut is distinguishable by $w = \lambda$.
3. Split a block: if two states of the same block go (on the same symbol) to different blocks, they are distinguishable – split accordingly. Repeat until no split possible.
4. Blocks = states of the minimal DFA (merge each block into one state).

Ex. 2.15 $F = \{q_0, q_1, q_3\}$, $\{q_2, q_4\}$ non-final. $\delta(q_0, 0) = q_1$ vs $\delta(q_1, 0) = q_2$ ($q_2 \notin F$): q_0, q_1 distinguishable by 0; split $\{q_0, q_1, q_3\} \to \{q_0\}, \{q_1, q_3\}$. Similarly $\delta(q_2, 0) = q_1$ vs $\delta(q_4, 0) = q_4$: split $\{q_2, q_4\} \to \{q_2\}, \{q_4\}$. Minimal DFA is unique (up to state renaming) – gives algorithm for checking DFA equivalence: minimize both, compare.

Ch 3 – Regular Expressions & Regular Grammars (Lec 3)

Regular Expressions

An RE is a *notation*, not a language: it *describes* a language (say "L is described by $(a + b + c)^*$", never "$L = (a + b + c)^*$").

Def. 3.1 Primitive: $\emptyset$, λ, and $a \in \Sigma$. If r_1, r_2 are REs, so are $r_1 + r_2$, $r_1 \cdot r_2$, r_1^*, (r_1). Nothing else. Semantics: $L(\emptyset) = \emptyset$; $L(\lambda) = \{\lambda\}$; $L(a) = \{a\}$; $L(r_1 + r_2) = L(r_1) \cup L(r_2)$; $L(r_1 r_2) = L(r_1)L(r_2)$; $L(r^*) = (L(r))^*$; $L((r)) = L(r)$. **Ex. 3.1** $(a + b \cdot c)^* \cdot (c + \emptyset)$ is an RE; $(a + b +)$ is not ($a^+ = aa^*$). **Ex. 3.2** $(a + b) \cdot a^*$ describes $\{a, aa, \dots, b, ba, \dots\}$. **Operator priority:** $* > \cdot > +$. $a \cdot b + c$ means $(a \cdot b) + c$: $\{ab, c\} \neq \{ab, ac\}$. **Ex. 3.3** $(a + b)^*(a + bb)$ = strings on $\{a, b\}$ ending in a or bb. **Ex. 3.4** $(aa)^*(bb)^*b = \{a^{2n}b^{2m+1} : n, m \geq 0\}$. **Ex. 3.5** contains 00: $(0 + 1)^*00(0 + 1)^*$. **Ex. 3.6** no consecutive 0s: $r_1 = (1 + 01)^*(0 + \lambda)$; also $r_2 = (1^*011^*)^*(0 + \lambda) + 1^*(0 + \lambda)$. Infinitely many REs per language. Equivalent REs: $L(r_1) = L(r_2)$. **More:** $L_2 = \{x : |x| \text{ even}\}$: $((0 + 1)(0 + 1))^*$; $L_3 = \{x : x$ does not end in $01\}$: $\lambda + 0 + 1 + (0 + 1)^*(00 + 10 + 11)$; L_4 = odd number of 0s: $1^*01^*(01^*01^*)^*$; exactly one a on $\{a, b, c\}$: $(b + c)^*a(b + c)^*$.

RE ⇔ Regular Languages (Kleene's theorem)

Thm. 3.1 ($\Rightarrow$) $L(r)$ regular for every RE r – induction on structure: primitives $\emptyset, \lambda, a$ have trivial NFAs; union/concatenation/star use the standard NFA constructions (with single final states).

Thm. 3.2 ($\Leftarrow$) every regular language has an RE – from DFA/NFA via **generalized transition graphs (GTG)**: edges labeled by REs; merge parallel edges with $+$; label missing edges $\emptyset$.

GTG state elimination (NFA → RE procedure):

1. Single final state (else add one, λ from old finals).
2. Complete the GTG (add $\emptyset$ edges).
3. If only q_0 and final q_f: $r = r_{ii}^* r_{ij} (r_{jj} + r_{ji} r_{ii}^* r_{ij})^*$.
4. Else pick q_k to remove; for each pair $p, q \in \{i, j\}$: replace $r_{pq} \mapsto r_{pq} + r_{pk} r_{kk}^* r_{kq}$; delete q_k.
5. Simplify with $r + \emptyset = r$, $r\emptyset = \emptyset$, $\emptyset^* = \lambda$.
6. Repeat until two states.

Ex. 3.8 GTG $q_0 \overset{a}{\circlearrowleft} \xrightarrow{b} q_1$, $q_1 \overset{a+b}{\circlearrowleft} \xrightarrow{b} q_2$: eliminate q_1: $q_0 \to q_2$: $bb^*(a + b)$; $q_0 \to q_0$: bb^*a; result $r = (bb^*a)^*bb^*(a + b)b^*$. **Ex. 3.11** $L = \{w : n_a(w)$ even, $n_b(w)$ odd$\}$: build 4-state NFA (EE,EO,OE,OO; start EE, final OE); eliminate to get RE (slides: from EE to OE via λ: with self-loops aa and bb; answer $(aa + bb)^* (ab + ba) (aa + bb)^* \cdot (\dots)$ – compute via elimination; practice!).

Regular Grammars

Def. linear grammar ≤ 1 variable on RHS of each production. ($S \to aSb$, $S \to \lambda$ linear.) **Def. right-linear** $A \to xB$ or $A \to x$; **Def. left-linear** $A \to Bx$ or $A \to x$ ($x \in T^*$). **Regular grammar** = right- or left-linear. **Every regular grammar is linear; not conversely.** G_3: $S \to A$, $A \to aB|\lambda$, $B \to Ab$ is linear but not regular; $L(G_3) = \{a^n b^n\}$. **Ex. 3.13** G_1: $S \to abS|a$ gives $L = (ab)^*a$. G_2: $S \to Aab$, $A \to Aab|B$, $B \to a$ gives $L = aab(ab)^*$. **Thm. 3.3,3.4** Regular grammars ⇔ regular languages. **Right-linear grammar → NFA:** states = variables $V_0, V_1, \dots$ + special final state V_F (start V_0). $V_i \to a_1 a_2 \cdots a_m V_j$: chain of m edges via new intermediate nodes to V_j. $V_i \to a_1 \cdots a_m$: chain to V_F. **Ex. 3.15** $V_0 \to aV_1$, $V_1 \to abV_0|b$: NFA $V_0 \xrightarrow{a} V_1$, $V_1 \xrightarrow{a} x \xrightarrow{b} V_0$, $V_1 \xrightarrow{b} V_F$; $L = (aab)^*ab$. **Left-linear grammar → right-linear:** reverse RHS strings: $A \to Ba_1 \cdots a_k \mapsto A \to a_k \cdots a_1 B$ ($= A \to v^R B$); $A \to a_1 \cdots a_k \mapsto A \to a_k \cdots a_1$. Then $L(G') = L(G)^R$; since $L(G')$ regular, $L(G)$ regular. **NFA → right-linear grammar (BFS):** transition $q \xrightarrow{a} p$ gives production $q \to ap$; final state q_f gives $q_f \to \lambda$. Then $L(G) = L(M)$.

Ex. NFA $q_0 \xrightarrow{a} q_1$, $q_1 \overset{b}{\circlearrowleft}$, $q_1 \xrightarrow{a} q_2$, $q_2 \xrightarrow{b} q_3$, $q_3 \xrightarrow{\lambda} q_1$, q_3 final: productions $q_0 \to aq_1$, $q_1 \to bq_1|aq_2$, $q_2 \to bq_3$, $q_3 \to q_1|\lambda$; $L = ab^*ab(b^*ab)^*$. **Summary:** four equivalent formalisms: DFA, NFA, RE, RG.

Ch 4 – Properties of Regular Languages (Lec 4)

Closure Properties

Regular languages are closed under: union, concatenation, star, complement, intersection, difference, reversal. Constructions assume NFAs with single final state.

- **Union** $L_1 \cup L_2$: new start with λ-edges into both M_1, M_2; new single final.
- **Concatenation** $L_1 L_2$: λ from M_1 final to M_2 start; final states of M_2.
- **Star** L_1^*: new start (also final, to include λ) $\xrightarrow{\lambda} M_1$, M_1 final $\xrightarrow{\lambda}$ back to M_1 start; plus λ to new final.
- **Reversal** L^R: reverse all transitions; initial state becomes final; old final(s) become the single new initial (add new start with λ to all old finals).
- **Complement** $\bar{L}$: requires a **DFA** (complete it!); swap $F \leftrightarrow Q - F$. ($L = \{a^n b : n \geq 0\}$ complemented: trap becomes final, etc.)
- **Intersection** $L_1 \cap L_2$: **product construction** $M = (Q_1 \times Q_2, \dots)$: $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$; $F = \{(p, q) : p \in F_1,\ q \in F_2\}$. (Also: DeMorgan $\overline{\bar{L}_1 \cup \bar{L}_2}$.)
- **Difference** $L_1 - L_2$: same product machine with $F = \{(p, q) : p \in F_1,\ q \notin F_2\}$ (equivalently $L_1 \cap \bar{L}_2$).

Ex. $L_1 = \{x :$ 00 not substring$\}$, $L_2 = \{x : x$ ends in $01\}$: product DFA on states $AX, \dots, CZ$; final for $\cap$: only $F_1 \times F_2$ pairs. **Def. 4.1 homomorphism** $h : \Sigma \to \Gamma^*$; extend $h(w) = h(a_1) \cdots h(a_n)$, $h(L) = \{h(w) : w \in L\}$. **Ex. 4.2** $h(a) = ab$, $h(b) = bbc$: $h(aba) = abbbcab$, $h(\{aa, aba\}) = \{abab, abbbcab\}$. **Ex. 4.3** L denoted $r = (a + b^*)(aa)^*$, $h(a) = dbcc, h(b) = bdc$: image denoted $(dbcc + (bdc)^*)(dbccdbcc)^*$. **Thm. 4.3** If L regular, $h(L)$ regular (substitute in the RE/transitions).

Elementary Decision Questions

Standard representations: DFA, NFA, RE, RG. Given DFA for L:

- **Membership** ($w \in L$?): simulate DFA on w; check final state.
- **Emptiness** ($L = \emptyset$?): any path from q_0 to a final state (graph reachability)? No ⇒ empty.
- **Finiteness** (L finite?): L infinite iff there is a walk from q_0 to a final state containing a *cycle*. (No cycle on any accepting path ⇒ finite.)
- **Equality** ($L_1 = L_2$?): check $(L_1 \cap \bar{L}_2) \cup (\bar{L}_1 \cap L_2) = \emptyset$ (i.e. both differences empty). Equivalent: minimize both DFAs, compare.

Pumping Lemma (identifying nonregular languages)

Pigeonhole principle: n pigeons, $m < n$ holes ⇒ some hole has ≥ 2 pigeons. In a DFA with m states, any walk labeled by $|w| \geq m$ visits $> m$ states ⇒ a state repeats. First repetition occurs within first m moves. **Thm. 4.8 (Pumping Lemma)** If L is an infinite regular language, $\exists$ integer m such that every $w \in L$, $|w| \geq m$, can be written $w = xyz$ with $|xy| \leq m$, $|y| \geq 1$, and $xy^i z \in L$ for all $i = 0, 1, 2, \dots$ **The game (adversary = Opponent O, you**

= Prover P):

1. O picks m (number of states).
2. P picks $w \in L$, $|w| \geq m$ (any w, but it must be *in* L).
3. O decomposes $w = xyz$ with $|xy| \leq m$, $|y| \geq 1$.
4. P picks i such that $xy^iz \notin L$. If such i exists for *every* decomposition, L is not regular.

Ex. 4.7 $L = \{a^nb^n : n \geq 0\}$ not regular. Given m, pick $w = a^mb^m$. Since $|xy| \leq m$, $y = a^k$, $k \geq 1$. Pump $i = 2$: $xy^2z = a^{m+k}b^m \notin L$. Contradiction. **Ex. 4.8** $L = \{ww^R : w \in \Sigma^*\}$ not regular. Pick $w = a^mb^mb^ma^m$ (in L); $y = a^k$; $i = 2$: $a^{m+k}b^2ma^m \notin L$ (not a palindrome form). *Pitfall:* picking a^{2m} lets opponent win ($y = aa$ pumps fine) – choose w that kills every allowed y. **Ex. 4.9** $L = \{w : n_a(w) < n_b(w)\}$ not regular: $w = a^mb^{m+1}$, $y = a^k$, $i = 2$: $a^{m+k}b^{m+1} \notin L$. **Ex. 4.10** $L = \{(ab)^na^k : n > k \geq 0\}$ not regular: $w = (ab)^{m+1}a^m$; y inside first m chars; $i = 0$: $(ab)^pb(ab)^qa^m$ or $(ab)^ma^m \notin L$. **Ex. 4.11** $L = \{a^{n^2} : n \geq 0\}$ not regular: $w = a^{m^2}$, $y = a^k, 1 \leq k \leq m$; $i = 0$: $a^{m^2-k} \notin L$ since $(m-1)^2 < m^2 - k < m^2$. **Ex. 4.12** $L = \{a^nb^kc^{n+k} : n, k \geq 0\}$ not regular: $w = a^mb^mc^{2m}$, $i = 0$: $a^{m-k}b^mc^{2m} \notin L$. (Or: homomorphism $h(a) = a, h(b) = a, h(c) = c$ maps L to $\{a^{n+k}c^{n+k}\}$.) **Ex. 4.13** $L = \{a^nb^l : n \neq l\}$ not regular: $w = a^{m!}b^{(m+1)!}$, $y = a^k, 1 \leq k \leq m$; need i with $m!+(i-1)k = (m+1)!$: $i = 1+\frac{m \cdot m!}{k}$, integer since $k \leq m$. (Also via $L \cap a^*b^*$.) **Ex.** $L = \{a^p : p \text{ prime}\}$ not regular: $w = a^p$, $p =$ smallest prime $\geq m$; $y = a^k$; choose $i - 1 = p$: $xy^{p+1}z = a^{p+pk} = a^{p(k+1)}$ composite. **Common pitfalls:** (1) Pumping lemma can *never* prove regularity (it is one-way; e.g. $\{a^ib^jc^k : i = 1 \Rightarrow j = k\}$ satisfies PL but is nonregular); (2) starting with $w \notin L$ (e.g. a^m for the primes language); (3) assuming a specific decomposition instead of handling *all* decompositions allowed by $|xy| \leq m$.

Ch 5 – Context-Free Grammars & Parsing (Lec 5)

Context-Free Grammars

Def. 5.1 CFG $G = (V, T, S, P)$; productions $A \to x$ with $A \in V$, $x \in (V \cup T)^*$: the substitution of A can happen *regardless of context.* **Def.** $L(G) = \{w \in T^* : S \overset{*}{\Rightarrow} w\}$; L is a CFL iff some CFG generates it. Regular $\subsetneq$ CFL: e.g. $\{a^nb^n\}$, $\{ww^R\}$ are CFL but not regular. **Ex. 5.1** $S \to aSa|bSb|\lambda$: $L = \{ww^R : w \in \{a,b\}^*\}$ (even palindromes). **Ex. 5.2** $S \to abB$, $A \to aaBb$, $B \to bbAa$, $A \to \lambda$: $L = \{ab(bbaa)^nbba(ba)^n : n \geq 0\}$. **Ex. 5.3** $\{a^nb^m : n \neq m\}$ is CFL: $S \to AS_1|S_1B$ (n>m / n<m), $S_1 \to aS_1b|\lambda$, $A \to aA|a$, $B \to bB|b$. (Not linear.) **Ex. 5.4** $S \to aSb|SS|\lambda$: $L = \{w : n_a(w) = n_b(w)$ and every prefix has $n_a \geq n_b\}$ = matched parentheses. (Note: adding $S \to bSa$ gives the unconstrained equal-count language.) **Derivation order:** leftmost = always replace leftmost variable; rightmost = always replace rightmost variable. In linear grammars the order is forced; in general CFGs it is not. **Ex. 5.5** $S \to aAB$, $A \to bBb$, $B \to A|\lambda$. Leftmost: $S \Rightarrow aAB \Rightarrow abBbB \Rightarrow abAbB \Rightarrow abbBbbB \Rightarrow abbbbB \Rightarrow abbbb$. Rightmost: $S \Rightarrow aAB \Rightarrow aA \Rightarrow abBb \Rightarrow abAb \Rightarrow abbBbb \Rightarrow abbbb$. **Def. 5.3 derivation (parse) tree** ordered tree: (1) root S; (2) leaves from $T \cup \{\lambda\}$; (3) internal vertices from V; (4) node A with children $a_1 \cdots a_n$ requires production $A \to a_1 \cdots a_n$; (5) a λ-leaf has no sibling. **Yield** = leaves read left-to-right. Partial derivation tree: properties 3–5, root may be any variable, leaves in $V \cup T \cup \{\lambda\}$; yield is a sentential form. **Thm. 5.1** $w \in L(G)$ iff there is a derivation tree with yield w; yield of any partial tree rooted at S is a sentential form. **One derivation tree $\leftrightarrow$ many derivations:** leftmost and rightmost derivations of aab from $S \to AB, A \to aaA|\lambda, B \to Bb|\lambda$ give the same tree.

Parsing

Parsing = given w, find a derivation sequence of productions. **Exhaustive search (top-down):** round 1: all $S \Rightarrow x$ one-step forms; round k: apply all productions to each form of round $k-1$ (e.g. $|P|^k$ forms). Flaws: tedious; may *not terminate* for $w \notin L(G)$ when λ or unit productions exist (sentential form length doesn't grow). **Ex. 5.7** $S \to SS|aSb|bSa|\lambda$, find $aabb$: phase 3 finds $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$. **Ex. 5.8** Replace by $S \to SS|aSb|bSa|ab|ba$ (no λ, no unit): each round lengthens the form, so parsing w ends within $|w|$ rounds. **Thm. 5.2** If CFG has no $A \to \lambda$ and no $A \to B$ productions, exhaustive search terminates for *any* w: $\leq 2|w|$ phases, $\leq |P| + |P|^2 + \cdots + |P|^{2|w|}$ forms $\Rightarrow O(|P|^{2|w|+1})$ – terrible. General CFG membership: $O(|w|^3)$ (CYK, Lec 6). **S-grammar (simple grammar):** productions $A \to ax$ (a terminal, $x \in V^*$) and each pair (A, a) occurs *once*. Parsing is deterministic: one choice per phase, total time $|w|$ (linear). **Ex. 5.9** $S \to aS|bSS|c$ is an s-grammar: $S \Rightarrow aS \Rightarrow abSS \Rightarrow abcS \Rightarrow abcc$ unique.

Ambiguity

Def. 5.5 CFG G is **ambiguous** iff some $w \in L(G)$ has ≥ 2 distinct derivation trees, equivalently ≥ 2 leftmost (or rightmost) derivations. **Ex. 5.11** $E \to E + E|E * E|(E)|a$: $a + a * a$ has two trees $((a + a) * a$ vs $a + (a * a))$ $\Rightarrow$ ambiguous. With values $2 + 2 * 2$: results 8 vs 6 – ambiguity is dangerous for PLs. **Fix (unambiguous precedence grammar):** $E \to E + T|T$, $T \to T * F|F$, $F \to (E)|a$. $*$ binds tighter than $+$; unique tree for $a + a * a$ giving $a + (a * a)$. **Dangling else:** IF_STMT → if E then S | if E then S else S is ambiguous (two trees for "if e1 then if e2 then s1 else s2"). Fix by matching else with nearest unmatched then. **Inherent ambiguity:** some CFLs have *only* ambiguous grammars, e.g. $L = \{a^nb^nc^m\} \cup \{a^nb^mc^m\}$: string $a^nb^nc^n$ always has two trees. (Deciding inherent ambiguity: hard.) **Ex. Ex 5.3.6** $S \to AB|aaB$, $A \to a|Aa$, $B \to b$ ambiguous: two leftmost derivations of aab: $S \Rightarrow aaB \Rightarrow aab$ and $S \Rightarrow AB \Rightarrow AaB \Rightarrow aaB \Rightarrow aab$. **CFGs and programming languages:** BNF (Backus-Naur form) $\langle expr \rangle ::= \langle term \rangle | \langle expr \rangle + \langle term \rangle$; LL and LR grammars parse in linear time. Syntactic (grammar) vs semantic (types) errors are separate.

Ch 6 – Grammar Transformations & Normal Forms (Lec 6)

Transformation Methods

Substitution rule **Thm. 6.1**: if $A \to x_1Bx_2$ and $B \to y_1|\cdots|y_n$ (all B-productions), replace by $A \to x_1y_1x_2|\cdots|x_1y_nx_2$ (optionally keep $A \to x_1Bx_2$). L unchanged. **Ex. 6.1** $A \to a|aaA|abBc$, $B \to abbA|b \Rightarrow A \to a|aaA|ababbAc|abbc$. **(1) Remove λ-productions** ($A \to \lambda$): find all *nullable* variables ($A \overset{*}{\Rightarrow} \lambda$); for each production, add all variants with each nullable variable kept or removed (skip the variant that deletes everything if it gives $A \to \lambda$ for non-start). Keep $S \to \lambda$ only if $\lambda \in L(G)$ (via extra start $S_0 \to S|\lambda$). **Ex.** $S \to aMb$, $M \to aMb|\lambda$: nullable $\{M\}$; result $S \to aMb|ab$, $M \to aMb|ab$. **Ex. 6.5** $S \to ABaC$, $A \to BC$, $B \to b|\lambda$, $C \to D|\lambda$, $D \to d$; nullable $\{A, B, C\}$: $S \to ABaC|BaC|AaC|ABa|aC|Aa|Ba|a$; $A \to B|C|BC$; $B \to b$; $C \to D$; $D \to d$. **(2) Remove unit-productions** ($A \to B$): remove $A \to A$ immediately; for each $B \to y_1|\cdots|y_n$ (non-unit), add $A \to y_1|\cdots|y_n$; iterate (use dependency graph). **Ex. 6.6** $S \to Aa|B$, $B \to A|bb$, $A \to a|bc|B$: dependency $A \to B \to A$; final $S \to a|bc|bb|Aa$, $A \to a|bb|bc$, $B \to a|bb|bc$. **(3) Remove useless productions:** variable useful iff $S \overset{*}{\Rightarrow} xAy \overset{*}{\Rightarrow} w \in T^*$. Two passes: (a) find variables that derive terminal strings (bottom-up marking), delete others *and their productions*; (b) draw dependency graph (edge $A \to B$ iff production $A \to xBy$); delete variables unreachable from S. Production useless iff it contains a useless variable. **Ex. 6.3** $S \to aS|A|C$, $A \to a$, $B \to aa$, $C \to aCb$: pass (a): $\{A, B\}$ derive terminals; $S \to A$ adds S; C never terminates $\Rightarrow$ delete C: $S \to aS|A$, $A \to a$, $B \to aa$. Pass (b): B unreachable $\Rightarrow$ delete: $S \to aS|A$, $A \to a$. **Thm. 6.5** If L is CFL without λ: exists CFG without useless/unit/λ-productions. **Removal order matters:** (1) λ-productions, (2) unit-productions, (3) useless productions.

Normal Forms

Def. CNF (Chomsky): all productions $A \to BC$ (two variables) or $A \to a$ (one terminal). (May need $S \to \lambda$ handled separately.) **Why CNF:** parse trees become binary; for a string of length n, tree depth $\geq \log n$; enables pumping lemma for CFLs and CYK. **Ex. 6.7** $S \to AS|a$, $A \to SA|b$ is CNF. ($S \to AAS$, $A \to aa$ are not.) **CNF procedure** **Thm. 6.6**:

1. Remove λ-productions, unit-productions, useless productions.
2. For each terminal a: add $T_a \to a$ and replace a by T_a in all productions with RHS length ≥ 2.
3. Replace $A \to C_1C_2 \cdots C_n$ ($n \geq 3$) by $A \to C_1V_1$, $V_1 \to C_2V_2$, $\ldots$, $V_{n-2} \to C_{n-1}C_n$.

Ex. 6.8 $S \to ABa,\ A \to aab,\ B \to Ac$: terminals: $T_a \to a, T_b \to b, T_c \to c$; $S \to ABT_a,\ A \to T_aT_aT_b,\ B \to AT_c$; binarize: $S \to AV_1,\ V_1 \to BT_a$; $A \to T_aV_2,\ V_2 \to T_aT_b$; $B \to AT_c$. **Def. GNF (Greibach)**: all productions $A \to aV_1 \cdots V_k,\ k \geq 0$ (terminal first, then only variables). **Ex.** $S \to cAB,\ A \to aA|bB|b,\ B \to b$ is GNF. $S \to abSb,\ S \to aa$ is not.

Ex. 6.9 $S \to AB,\ A \to aA|bB|b,\ B \to b \Rightarrow S \to aAB|bBB|bB$.

Ex. 6.10 $S \to abSb|aa \Rightarrow S \to aT_bST_b|aT_a,\ T_a \to a,\ T_b \to b$.

Thm. 6.7 Every CFG without λ has an equivalent GNF. **Observation:** GNF is excellent for parsing but hard to compute; CNF is easy to find.

CYK Membership Algorithm

Input: CFG in CNF, string w. Output: is $w \in L(G)$? **Dynamic programming over substrings:** let $V_{ij} = \{A : A \overset{*}{\Rightarrow} w_i \cdots w_j\}$ (variables deriving substring $w_i \cdots w_j$).

- **Base** (length 1): $V_{ii} = \{A : A \to w_i \in P\}$.
- **Induction** (length > 1): for all k, $i \leq k < j$: if $A \to BC \in P$, $B \in V_{ik}$, $C \in V_{k+1,j}$, then add A to V_{ij}.
- $w \in L(G)$ iff $S \in V_{1n}$.

Time: $O(|w|^3)$ (positions $O(n^2)$, splits $O(n)$); also converts to a bottom-up parser. **Ex.** CNF $S \to AB,\ A \to BB|a,\ B \to AB|b$; $w = aabbb$: length-1: $V_{11} = V_{22} = \{A\}$, $V_{33} = V_{44} = V_{55} = \{B\}$. length-2: $aa \mapsto \emptyset$; $ab \mapsto \{S, B\}$ (both $S \to AB$ and $B \to AB$ with $A \in V_{11}, B \in V_{33}$); $bb \mapsto \{A\}$ ($A \to BB$), so $V_{34} = V_{45} = \{A\}$. length-3: $aab \mapsto \{S, B\}$; $abb \mapsto \{A\}$; $bbb \mapsto \{S, B\}$. length-4: $aabb \mapsto \{A\}$; $abbb \mapsto \{S, B\}$. length-5: $aabbb \mapsto \{S, B\} \ni S \Rightarrow aabbb \in L(G)$. **Layout as a triangle** above the string (as in slides) with cells V_{ij}.

Quick Reference: Theorems & Definitions

Def 2.1	DFA $M = (Q, \Sigma, \delta, q_0, F)$, total δ
Def 2.5/2.6	NFA: $\delta : Q \times (\Sigma \cup \{\lambda\}) \to 2^Q$; $L(M) = \{w : \delta^*(q_0, w) \cap F \neq \emptyset\}$
Thm 2.2	NFA languages = regular languages (subset construction)
Def 2.8	indistinguishable states $\Rightarrow$ DFA minimization
Def 3.1	REs; primitive $\emptyset, \lambda, a$; $+, \cdot, ^*$
Thm 3.1/3.2	RE $\Leftrightarrow$ regular (Kleene): NFA constructions / GTG elimination
Thm 3.3/3.4	Regular grammars $\Leftrightarrow$ regular languages
Thm 4.8	Pumping Lemma: $w = xyz$, $\|xy\| \leq m$, $\|y\| \geq 1$, $xy^iz \in L\ \forall i$
Thm 4.3	Homomorphic image of regular language is regular
Def 5.1	CFG $G = (V, T, S, P)$, $A \to x$, $x \in (V \cup T)^*$
Def 5.5	ambiguity: ≥ 2 trees / leftmost derivations
Thm 6.1	substitution rule preserves $L(G)$
Thm 6.5	remove λ, then unit, then useless productions
Thm 6.6	every CFG (no λ) $\to$ CNF
Thm 6.7	every CFG (no λ) $\to$ GNF
CYK	membership in CNF, dynamic programming, $O(n^3)$